

以数传 SDK 2.1.0 hid br23 board_ac635n_demo 为例：

1. 首先要更新文件里的 `cpu.a`
2. 在板级头文件 `board_ac635n_demo_cfg.h` 添加 SD 卡相关的配置：

```
89 //*****//
90 //                      SD 配置                      //
91 //*****//
92 #define SD_CMD_DECT      0
93 #define SD_CLK_DECT      1
94 #define SD_IO_DECT       2
95 #define TCFG_SD0_ENABLE      DISABLE_THIS_MOUDLE
96 //A组IO: CMD:PA9   CLK:PA10   DAT0:PA5   DAT1:PA6   DAT2:PA7   DAT3:PA8
97 //B组IO: CMD:PB10  CLK:PB9   DAT0:PB8   DAT1:PB6   DAT2:PB5   DAT3:PB4
98 #define TCFG_SD0_PORTS      'B'
99 #define TCFG_SD0_DAT_MODE    1
100 #define TCFG_SD0_DET_MODE    SD_CMD_DECT
101 #define TCFG_SD0_DET_IO      IO_PORT_DM//当SD_DET_MODE为2时有效
102 #define TCFG_SD0_DET_IO_LEVEL 0//IO检查, 0: 低电平检测到卡。 1: 高电平(外部电源)检测到卡。 2: 高电平(SD卡电源)检测到卡。
103 #define TCFG_SD0_CLK         (3000000*4L)
104
105 #define TCFG_SD1_ENABLE      DISABLE_THIS_MOUDLE
106 //A组IO: CMD:PC4   CLK:PC5   DAT0:PC3   DAT1:PC2   DAT2:PC1   DAT3:PC0
107 //B组IO: CMD:PB5   CLK:PB6   DAT0:PB4   DAT1:PB8   DAT2:PB9   DAT3:PB10
108 #define TCFG_SD1_PORTS      'A'
109 #define TCFG_SD1_DAT_MODE    1
110 #define TCFG_SD1_DET_MODE    SD_CLK_DECT
111 #define TCFG_SD1_DET_IO      IO_PORT_DM//当SD_DET_MODE为2时有效
112 #define TCFG_SD1_DET_IO_LEVEL 0//IO检查, 0: 低电平检测到卡。 1: 高电平(外部电源)检测到卡。 2: 高电平(SD卡电源)检测到卡。
113 #define TCFG_SD1_CLK         (3000000*4L)
114
115 //*****//
116 //                      SD 配置                      //
117 //*****//
118
119 #define SD_CMD_DECT      0
120 #define SD_CLK_DECT      1
121 #define SD_IO_DECT       2
122
123 #define TCFG_SD0_ENABLE      DISABLE_THIS_MOUDLE
124 //A 组 IO: CMD:PA9   CLK:PA10   DAT0:PA5   DAT1:PA6   DAT2:PA7   DAT3:PA8
125 //B 组 IO: CMD:PB10  CLK:PB9   DAT0:PB8   DAT1:PB6   DAT2:PB5   DAT3:PB4
126 #define TCFG_SD0_PORTS      'B'
127 #define TCFG_SD0_DAT_MODE    1
128 #define TCFG_SD0_DET_MODE    SD_CMD_DECT
129 #define TCFG_SD0_DET_IO      IO_PORT_DM//当 SD_DET_MODE 为 2 时有效
130 #define TCFG_SD0_DET_IO_LEVEL 0//IO 检查, 0: 低电平检测到卡。 1: 高电平
131 (外部电源)检测到卡。 2: 高电平(SD 卡电源)检测到卡。
132 #define TCFG_SD0_CLK         (3000000*4L)
133
134
135 #define TCFG_SD1_ENABLE      DISABLE_THIS_MOUDLE
136 //A 组 IO: CMD:PC4   CLK:PC5   DAT0:PC3   DAT1:PC2   DAT2:PC1   DAT3:PC0
137 //B 组 IO: CMD:PB5   CLK:PB6   DAT0:PB4   DAT1:PB8   DAT2:PB9   DAT3:PB10
138 #define TCFG_SD1_PORTS      'A'
139 #define TCFG_SD1_DAT_MODE    1
140 #define TCFG_SD1_DET_MODE    SD_CLK_DECT
141 #define TCFG_SD1_DET_IO      IO_PORT_DM//当 SD_DET_MODE 为 2 时有效
142 #define TCFG_SD1_DET_IO_LEVEL 0//IO 检查, 0: 低电平检测到卡。 1: 高电平
143 (外部电源)检测到卡。 2: 高电平(SD 卡电源)检测到卡。
144 #define TCFG_SD1_CLK         (3000000*4L)
```

3. 在板级的 C 文件 board_ac635n_demo.c 添加 SD 卡相关代码

```
9 #include "asm/iic_soft.h"
10 #include "usb/otg.h"
11 #include "asm/sdmmc.h"
12
13 #ifdef CONFIG_LITE_AUDIO

169 /***** SD config *****/
170 #if TCFG_SD0_ENABLE
171 SD0_PLATFORM_DATA_BEGIN(sd0_data)
172     .port                = TCFG_SD0_PORTS, //sd0 support port 'A' and port 'B'
173     .data_width          = TCFG_SD0_DAT_MODE,
174     .speed               = TCFG_SD0_CLK,
175     .detect_mode         = TCFG_SD0_DET_MODE,
176
177     .priority            = 3,
178     #if (TCFG_SD0_DET_MODE == SD_IO_DECT)
179     .detect_io           = TCFG_SD0_DET_IO, //当检测模式为io口检测是有效
180     .detect_io_level     = TCFG_SD0_DET_IO_LEVEL, //0: 低电平检测到卡。 1: 高电平
181     .detect_func         = sdmmc_0_io_detect, //库函数，需要detect_io和detect_io_
182     .power               = sd_set_power, //库函数，使用SDPG引脚。可以自行重写其他
183     /* .power            = NULL, */
184     #elif (TCFG_SD0_DET_MODE == SD_CLK_DECT)
185     .detect_io_level     = TCFG_SD0_DET_IO_LEVEL, //0: 低电平检测到卡。 1: 高电平
186     .detect_func         = sdmmc_0_clk_detect, //库函数，需要detect_io_level元素作
187     .power               = sd_set_power, //库函数，使用SDPG引脚。可以自行重写其他
188     /* .power            = NULL, */
189     #else
190     .detect_func         = sdmmc_cmd_detect,
191     .power               = NULL, //cmd检测需要全程供电，建议用硬件固定电源。当然，
192     /* .power            = sd_set_power, */
193     #endif
194 SD0_PLATFORM_DATA_END()
195 #endif
196
197 #if TCFG_SD1_ENABLE
198 SD1_PLATFORM_DATA_BEGIN(sd1_data)
199     .port                = TCFG_SD1_PORTS, //sd1 only support port 'A'
200     .data_width          = TCFG_SD1_DAT_MODE,
201     .speed               = TCFG_SD1_CLK,
202     .detect_mode         = TCFG_SD1_DET_MODE,
203
204     .priority            = 3,
205     #if (TCFG_SD1_DET_MODE == SD_IO_DECT)
206     .detect_io           = TCFG_SD1_DET_IO, //当检测模式为io口检测是有效
207     .detect_io_level     = TCFG_SD1_DET_IO_LEVEL, //0: 低电平检测到卡。 1: 高电平
208     .detect_func         = sdmmc_1_io_detect, //库函数，需要detect_io和detect_io_
209     .power               = sd_set_power, //库函数，使用SDPG引脚。可以自行重写其他
210     /* .power            = NULL, */
211     #elif (TCFG_SD1_DET_MODE == SD_CLK_DECT)
212     .detect_io_level     = TCFG_SD1_DET_IO_LEVEL, //0: 低电平检测到卡。 1: 高电平
213     .detect_func         = sdmmc_1_clk_detect, //库函数，需要detect_io_level元素作
214     .power               = sd_set_power, //库函数，使用SDPG引脚。可以自行重写其他
215     /* .power            = NULL, */
216     #else
217     .detect_func         = sdmmc_cmd_detect,
218     .power               = NULL, //cmd检测需要全程供电，建议用硬件固定电源。当然，
219     /* .power            = sd_set_power, */
220     #endif
221 SD1_PLATFORM_DATA_END()
222 #endif

/**** SD config *****/
#if TCFG_SD0_ENABLE
SD0_PLATFORM_DATA_BEGIN(sd0_data)
    .port                = TCFG_SD0_PORTS, //sd0 support port 'A' and port 'B'
    .data_width          = TCFG_SD0_DAT_MODE,
    .speed               = TCFG_SD0_CLK,
```

```

        .detect_mode                = TCFG_SD0_DET_MODE,

        .priority                    = 3,
    #if (TCFG_SD0_DET_MODE == SD_IO_DECT)
        .detect_io                  = TCFG_SD0_DET_IO,//当检测模式为 io 口检测是有效
        .detect_io_level            = TCFG_SD0_DET_IO_LEVEL,//0: 低电平检测到卡。 1: 高电平
检测到卡
        .detect_func                = sdmmc_0_io_detect,//库函数，需要 detect_io 和
detect_io_level 两个元素作为参数。可以自行重写一个检测函数，在线返回 1，不在线返回 0，
即可。
        .power                      = sd_set_power,//库函数，使用 SDPG 引脚。可以自行重写其
他的 SD 卡电源控制函数，传入 0: 关电源。传入 1，开电源。如果电源硬件已固定不可控，
则该函数无效，可以填 NULL
        /* .power                    = NULL, */
    #elif (TCFG_SD0_DET_MODE == SD_CLK_DECT)
        .detect_io_level            = TCFG_SD0_DET_IO_LEVEL,//0: 低电平检测到卡。 1: 高电平
检测到卡
        .detect_func                = sdmmc_0_clk_detect,//库函数，需要 detect_io_level 元素作
为参数。可以自行重写一个检测函数，在线返回 1，不在线返回 0，即可。
        .power                      = sd_set_power,//库函数，使用 SDPG 引脚。可以自行重写其
他的 SD 卡电源控制函数，传入 0: 关电源。传入 1，开电源。如果电源硬件已固定不可控，
则该函数无效，可以填 NULL
        /* .power                    = NULL, */
    #else
        .detect_func                = sdmmc_cmd_detect,
        .power                      = NULL,//cmd 检测需要全程供电，建议用硬件固定电源。当
然，可以自行写其他的 SD 卡电源控制函数，传入 0: 关电源。传入 1，开电源。
        /* .power                    = sd_set_power, */
    #endif
SD0_PLATFORM_DATA_END()
#endif

#if TCFG_SD1_ENABLE
SD1_PLATFORM_DATA_BEGIN(sd1_data)
    .port                          = TCFG_SD1_PORTS, //sd1 only support port 'A'
    .data_width                    = TCFG_SD1_DAT_MODE,
    .speed                         = TCFG_SD1_CLK,
    .detect_mode                   = TCFG_SD1_DET_MODE,

    .priority                      = 3,
    #if (TCFG_SD1_DET_MODE == SD_IO_DECT)
        .detect_io                  = TCFG_SD1_DET_IO,//当检测模式为 io 口检测是有效
        .detect_io_level            = TCFG_SD1_DET_IO_LEVEL,//0: 低电平检测到卡。 1: 高电平
检测到卡

```

.detect_func = sdmmc_1_io_detect, // 库函数，需要 detect_io 和 detect_io_level 两个元素作为参数。可以自行重写一个检测函数，在线返回 1，不在线返回 0，即可。

.power = sd_set_power, // 库函数，使用 SDPG 引脚。可以自行重写其他的 SD 卡电源控制函数，传入 0：关电源。传入 1，开电源。如果电源硬件已固定不可控，则该函数无效，可以填 NULL

```
/* .power = NULL, */
#elif (TCFG_SD1_DET_MODE == SD_CLK_DECT)
    .detect_io_level = TCFG_SD1_DET_IO_LEVEL, // 0：低电平检测到卡。 1：高电平检测到卡
```

.detect_func = sdmmc_1_clk_detect, // 库函数，需要 detect_io_level 元素作为参数。可以自行重写一个检测函数，在线返回 1，不在线返回 0，即可。

.power = sd_set_power, // 库函数，使用 SDPG 引脚。可以自行重写其他的 SD 卡电源控制函数，传入 0：关电源。传入 1，开电源。如果电源硬件已固定不可控，则该函数无效，可以填 NULL

```
/* .power = NULL, */
#else
    .detect_func = sdmmc_cmd_detect,
    .power = NULL, // cmd 检测需要全程供电，建议用硬件固定电源。当然，可以自行写其他的 SD 卡电源控制函数，传入 0：关电源。传入 1，开电源。
```

```
/* .power = sd_set_power, */
```

```
#endif
```

```
SD1_PLATFORM_DATA_END()
```

```
#endif
```

4. 在板级的 C 文件 board_ac635n_demo.c，将卡接口注册到 dev 机制

```
236 #endif
237 REGISTER_DEVICES(device_table) = {
238     #if TCFG_SD0_ENABLE
239         { "sd0", &sd_dev_ops, (void *) &sd0_data},
240     #endif
241     #if TCFG_SD1_ENABLE
242         { "sd1", &sd_dev_ops, (void *) &sd1_data},
243     #endif
244     #if TCFG_OTG_MODE
245         { "otg", &usb_dev_ops, (void *) &otg_data},
246     #endif
247 };
248 void debug_uart_init(const struct uart_platform_data *data)
249 {
250     #if TCFG_UART0_ENABLE
```

```
#if TCFG_SD0_ENABLE
    { "sd0", &sd_dev_ops, (void *) &sd0_data},
```

```
#endif
```

```
#if TCFG_SD1_ENABLE
    { "sd1", &sd_dev_ops, (void *) &sd1_data},
```

```
#endif
```

5. 在 lib_driver_config.c 添加卡驱动相关的 const 变量

```
| 43  
| 44 #if TCFG_SD0_SD1_USE_THE_SAME_HW  
| 45 const int sd0_sd1_use_the_same_hw = 1;  
| 46 #else  
| 47 const int sd0_sd1_use_the_same_hw = 0;  
| 48 #endif  
| 49  
| 50 #if TCFG_KEEP_CARD_AT_ACTIVE_STATUS  
| 51 const int keep_card_at_active_status = 1;  
| 52 #else  
| 53 const int keep_card_at_active_status = 0;  
| 54 #endif  
| 55  
| 56 #if TCFG_SDX_CAN_OPERATE_MMC_CARD  
| 57 const int sdx_can_operate_mmc_card = 1;  
| 58 #else  
| 59 const int sdx_can_operate_mmc_card = 0;  
| 60 #endif  
| 61
```

```
#if TCFG_SD0_SD1_USE_THE_SAME_HW  
const int sd0_sd1_use_the_same_hw = 1;  
#else  
const int sd0_sd1_use_the_same_hw = 0;  
#endif
```

```
#if TCFG_KEEP_CARD_AT_ACTIVE_STATUS  
const int keep_card_at_active_status = 1;  
#else  
const int keep_card_at_active_status = 0;  
#endif
```

```
#if TCFG_SDX_CAN_OPERATE_MMC_CARD  
const int sdx_can_operate_mmc_card = 1;  
#else  
const int sdx_can_operate_mmc_card = 0;  
#endif
```


6. 在 lib_driver_config.c 最下面添加 SD 驱动的打印控制变量

```
215 const char log_tag_const_w_USB AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIB(TRUE);
216 const char log_tag_const_e_USB AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIB(TRUE);
217
218 const char log_tag_const_v_SD AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIB(FALSE);
219 const char log_tag_const_i_SD AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIB(TRUE);
220 const char log_tag_const_d_SD AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIB(FALSE);
221 const char log_tag_const_w_SD AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIB(FALSE);
222 const char log_tag_const_e_SD AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIB(TRUE);
223
```

```
const char log_tag_const_v_SD AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIB(FALSE);
const char log_tag_const_i_SD AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIB(TRUE);
const char log_tag_const_d_SD AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIB(FALSE);
const char log_tag_const_w_SD AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIB(FALSE);
const char log_tag_const_e_SD AT(.LOG_TAG_CONST) = CONFIG_DEBUG_LIB(TRUE);
```

7. 移好上述代码之后，板级根据原理图使能 SD 卡模块，就可以操作 SD 的物理扇区了
操作 SD 卡要在 devices_init(); 函数之后。

```
331
332 extern void set_dcdc_ctl_port(u8 port);
333 extern void cfg_file_parse(u8 idx);
334 void board_init()
335 {
336     board_power_init();
337     adc_vbg_init();
338     adc_init();
339     cfg_file_parse(0);
340     devices_init();
341
342     my_sd_test();
343
344     board_devices_init();
345
346 #if TCFG_CHARGE_ENABLE
347     charge_api_init((void *)&charge_data);
348 #if TCFG_HANDSHAKE_ENABLE
349     if(get_charge_online_flag()){
```

卡驱动的部分初始化

8. 以下是操作 SD 卡的参考示例:

```
298
299 static struct device *sd_test_hdl = NULL;
300 static u32 card_capacity = 0;
301 static u8 sd_test_buf[512] __attribute__((aligned(32)));
302
303 void my_sd_test(void)
304 {
305     sd_test_hdl = force_open_sd("sd1");//不检查卡的在线状态，直接强行打开设备，如果设备真不在线，则失败
306     if (!sd_test_hdl) {
307         printf("force open sd failed ! \n");
308         return;
309     }
310     //获取卡容量
311     dev_ioctl(sd_test_hdl, IOCTL_GET_CAPACITY, (u32)&card_capacity);
312     printf("card_capacity = %d\n", card_capacity);
313     do {
314         //操作前 设备恢复
315         dev_ioctl(sd_test_hdl, IOCTL_CMD_RESUME, 1);
316         len = dev_bulk_read(sd_test_hdl, sd_test_buf, /*扇区地址*/ 0, /*读多少个扇区*/ 4);
317         if (len == 0) { //读失败
318             break;
319         }
320         len = dev_bulk_write(sd_test_hdl, sd_test_buf, /*扇区地址*/ 0, /*写多少个扇区*/ 4);
321         if (len == 0) { //写失败
322             break;
323         }
324         //操作后，设备挂起（卡会进入idle，可以省一点功耗）
325         dev_ioctl(sd_test_hdl, IOCTL_CMD_SUSPEND, 1);
326     } while(0);
327
328     dev_close(sd_test_hdl);
329     sd_test_hdl = NULL;
330 }
```

```
static struct device *sd_test_hdl = NULL;
```

```
static u32 card_capacity = 0;
```

```
static u8 sd_test_buf[512] __attribute__((aligned(32)));
```

```
void my_sd_test(void)
```

```
{
```

```
    sd_test_hdl = force_open_sd("sd1");//不检查卡的在线状态，直接强行打开设备，如果设备真不在线，则失败
```

```
    if (!sd_test_hdl) {
```

```
        printf("force open sd failed ! \n");
```

```
        return;
```

```
    }
```

```
    //获取卡容量
```

```
    dev_ioctl(sd_test_hdl, IOCTL_GET_CAPACITY, (u32)&card_capacity);
```

```
    printf("card_capacity = %d\n", card_capacity);
```

```
    do {
```

```
        //操作前 设备恢复
```

```
        dev_ioctl(sd_test_hdl, IOCTL_CMD_RESUME, 1);
```

```
        len = dev_bulk_read(sd_test_hdl, sd_test_buf, /*扇区地址*/ 0, /*读多少个扇区*/ 4);
```

```
        if (len == 0) { //读失败
```

```
            break;
```

```
        }
```

```
        len = dev_bulk_write(sd_test_hdl, sd_test_buf, /*扇区地址*/ 0, /*写多少个扇区*/ 4);
```

```
        if (len == 0) { //写失败
```

```
            break;
```

```
        }
```

```
        //操作后，设备挂起（卡会进入 idle，可以省一点功耗）
        dev_ioctl(sd_test_hdl, IOCTL_CMD_SUSPEND, 1);
    } while(0);

    dev_close(sd_test_hdl);
    sd_test_hdl = NULL;
}
```